Hochschule Konstanz
Fakultät Informatik

Software Architecture

Microservices

Prof. Dr. Marko Boger

Monolithic Deployment

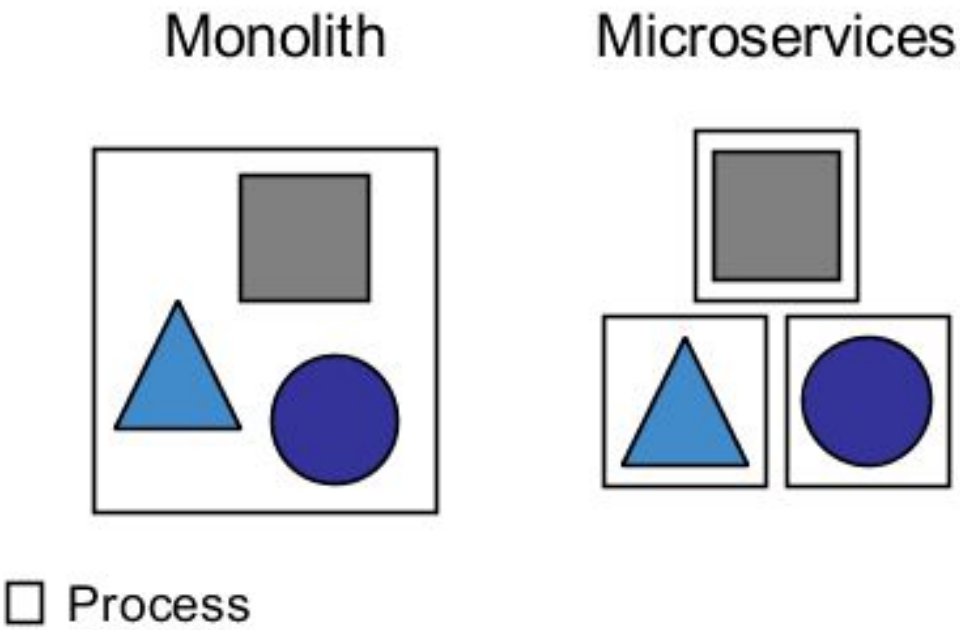
We have carefully developed a layered, component-based architecture with Dependency Injection.

- We can add new views or replace existing ones without changing the other layers
- We have interfaces in between layers and components and can replace implementations of components via DI
- Components can be replaced at compile time

- However, in terms of deployment, we so far have one deployment unit
  - in this sense, we have a Monolith

Deploy each Component individually

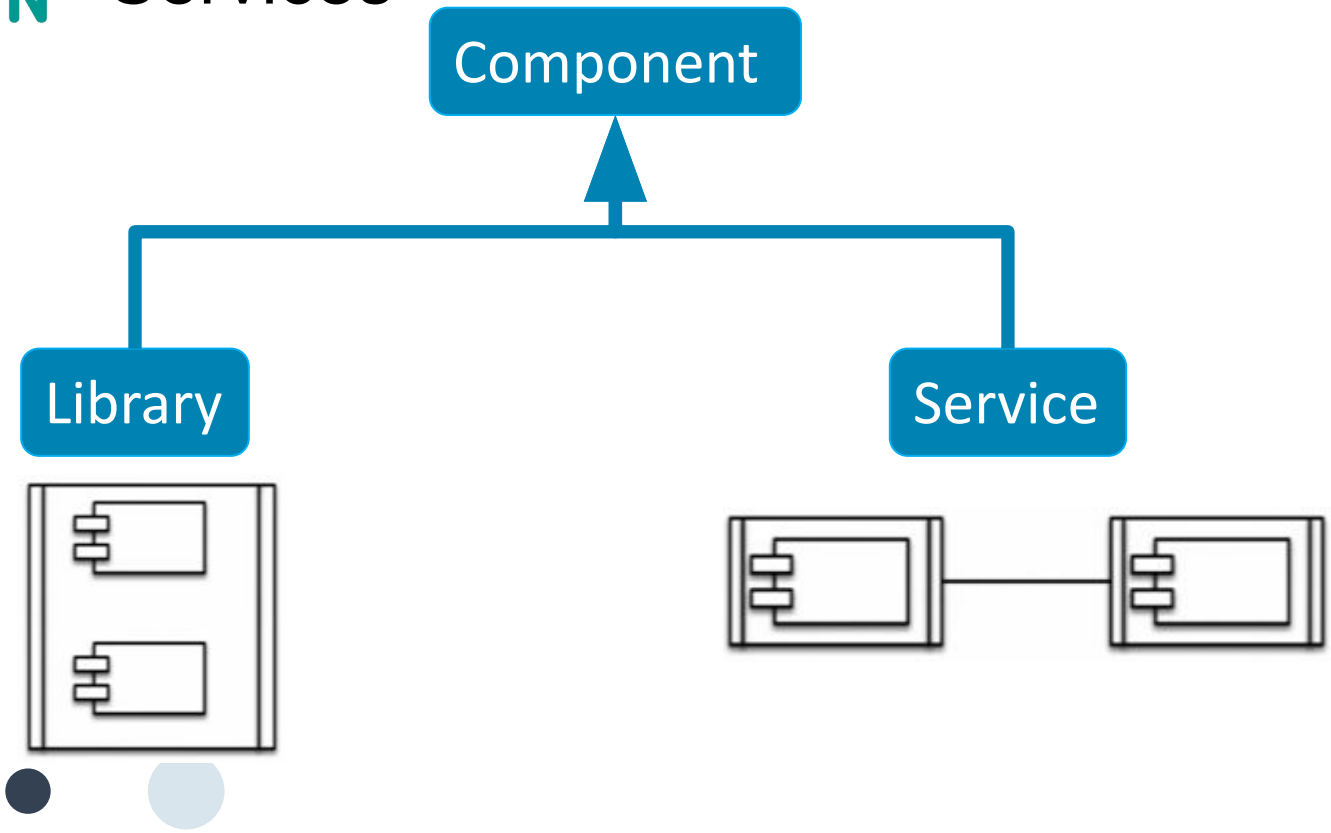
We now want to deploy each component individually.



Characteristics of Microservices

- Componentization via services
- Organized around business capabilities
- Products not Projects
- Smart endpoints and dumb pipes
- Decentralized Governance
- Decentralized Data Management
- Infrastructure Automation
- Design for Failure
- Evolutionary Design

Componentization via Services



Library vs Service

Library

- Multiple libraries are part of the process
- In-Memory calls
- Replacability & upgradability at compile time

Service

- Run in own process
- Communication via InterProcessCommunication (HTTP Web-Services)
- Independently replacable
- Independently upgradable

Organized around Business Capabilities

Classic approach

- Focus on Technology

Microservice approach

- Focus on business capability

Ui

Server

DB

Ordering

Delivery

Catalogue

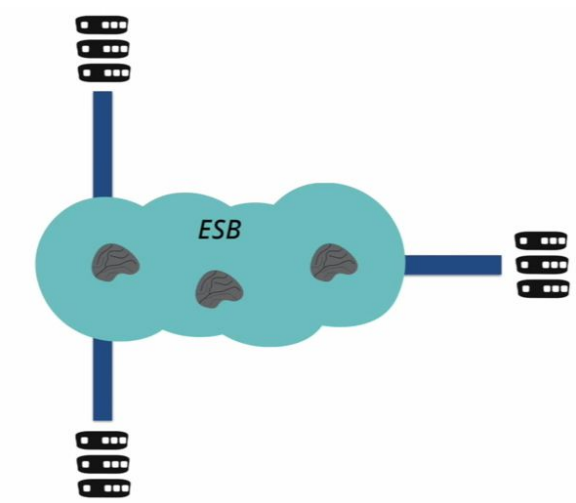
Products not Projects

- Classic development approach:
  - Project model
  - On completion, handed over
  - Maintenance organization takes responsibility
- Microservice approach:
  - Development teams should own a product over its full lifetime
  - Inspiration is Amazon‘s „You build it, you run it“
- Implications for software developers:
  - Day-to-day contact with how the software behaves in the wild
  - Increased contact with users
  - Product mentality overlaps with the characteristic of organisation around business capabilities

Smart endpoints and dumb pipes

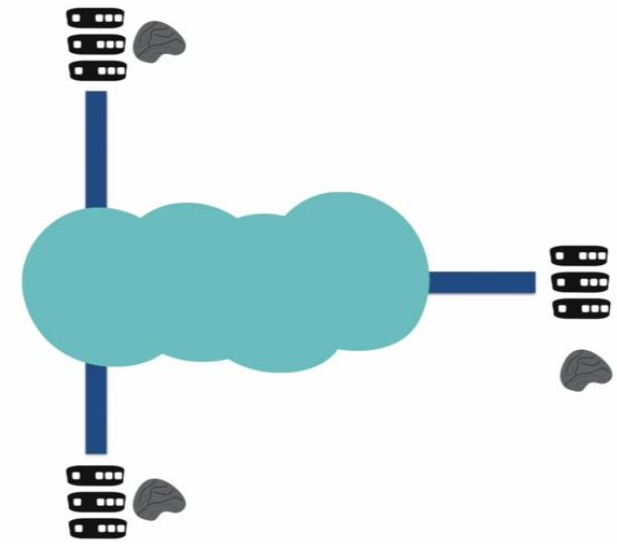
SOA approach

- Enterprise Service Bus
- Smart Cloud
  - Message routing
  - Choreography
  - Transformation
  - Applying business logic



Microservices approach

- Smart endpoints
- Receive request - Apply logic - Send response
- RESTfull protocolls



Decentralized Governance

Tendency to standardise on single technology platforms ->

Best technology for each component

C++

Node.js

Scala

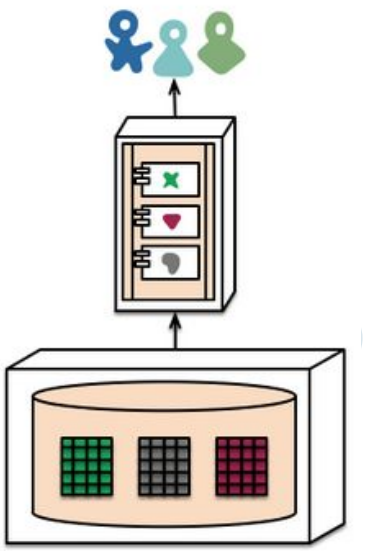
Decentralized Data Management

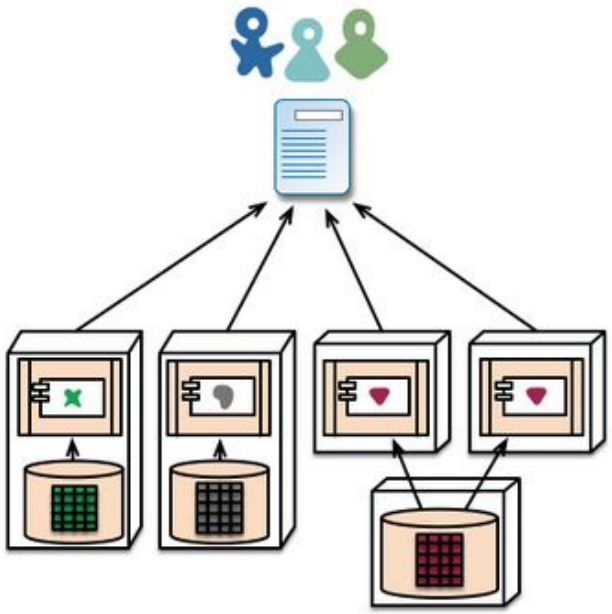
Monolithic approach:

- One DB
- One DB-technology

Microservice approach

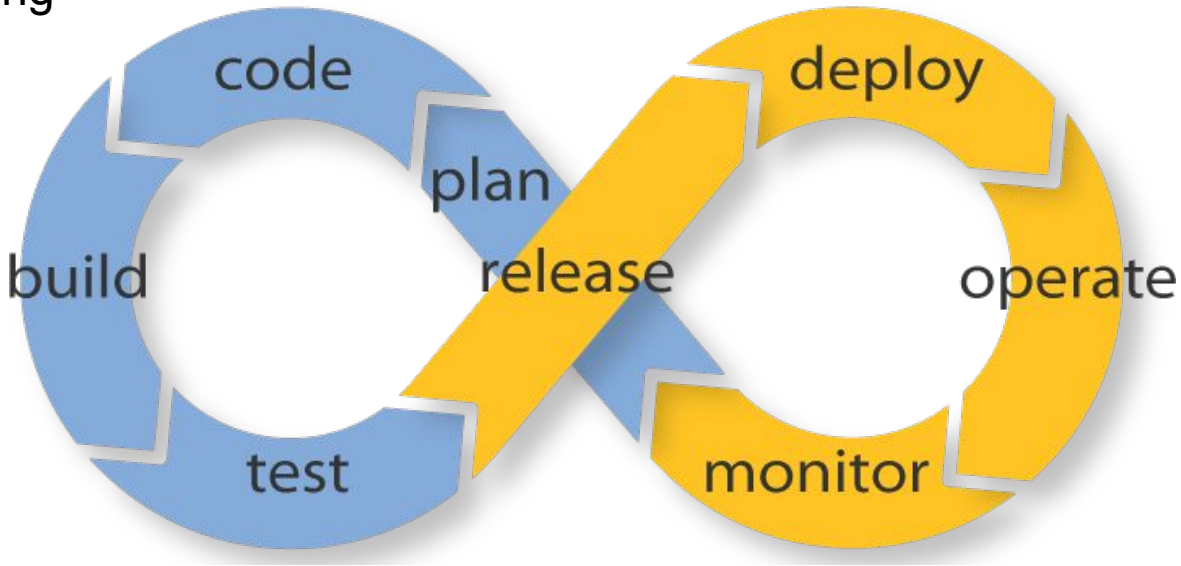
- most fitting database technology
- Polyglot Persistence





Infrastructure Automation

- Automated Testing
- Continuous Integration
- Continuous Deployment
- Monitoring



Design for Failure

Services can fail

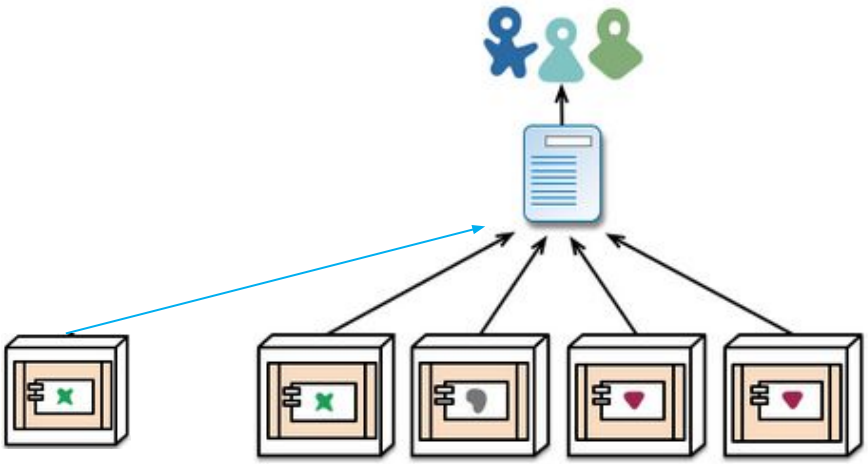
Assume failure during development and testing

Strong emphasis on real-time monitoring:

- architectural elements
- business relevant metrics

Evolutionary Design

- Componentization:
  - Independent replacement
  - Independent upgradeability
- Microservices:
  - Expect services to be scrapped or replaced



Advantages

Monolith

- Simplicity
- Consistency
- Inter-module refactoring

Microservice

- Partial deployment
- Availability
- Preserve modularity
- Multiple platforms
- Versioned independently

Use Case: Spotify

Size

- 90 Teams, 600 developers
- 5 Development offices
- 1 Product
- 810 Services actively running
- 10 Services per squad
- 1.7 Services per developer

Use Case:Netflix

- 81.5 million subscriber in 190 countries
- Over 1 Petabyte of data
- 125 Million hours / day of streaming
- 1000+ devices
- 37% of Internet traffic at peak
- Billions of Requests per day
  - 100.000s of customer interactions per minute

The Simian Army

- Induce delays and errors
- Terminate instances
- Security violations
- Zone outage
- Region outage

SBT Multi-Projects

SBT supports multiple projects very well.

This article shows how:

https://www.scala-sbt.org/1.x/docs/Multi-Project.html

Your sbt structure should look something like this:

```
lazy val root = (project in file("."))
  .aggregate(util, core)

lazy val util = (project in file("util"))

lazy val core = (project in file("core"))
```

Task 3

Split up your Monolith into Modules

- We will not go to full services in one step, but three.
- First step is to split up the Project into Modules.
- The Modules can have dependencies to other Modules.
- Step 2 will be REST
- Step 3 will be Docker

Find good cuts for your Application